

Lab 1.

제출일	2025.12.17	학번	
전공	전자공학과	이름	차상훈

문 제	▪ [1번 문제]: Compute the Z-transform of the finite-length signal $x[n] = a^n \cdot u[n]$ ($n = 0, 1, \dots, 50$), and compare it with the closed-form expression $X(z) = \frac{1}{1-az^{-1}}$
주 요 변 수	r, w, n,
알 고 리 즘	1. zTransform1D 함수 정의 <pre>#include <iostream> #include <vector> #include <complex> #include <cmath> #define P! 3.141592 using namespace std; complex<double> zTransform1D(const vector<double>& x, complex<double> z) { complex<double> X(0.0, 0.0); // X(z)=0 + j0 로 초기화 complex<double> z_pow(1.0, 0.0); // 처음에는 z^{-0}=1// 처음에는 z^{-0}=1 // n=0, 1, ..., x.size()-1 까지 반복 for (size_t n = 0; n < x.size(); ++n) { X += x[n] * z_pow; // x[n]*z^{-n} z_pow /= z; // z_pow = z_pow / z; } return X; }</pre> 위 코드는 1차원 z-Transform 함수를 구현한 것이며, 기본적으로 $x[n]$이 현실 세계에서 가장 보편적인 right-sided임을 가정하고 알고리즘을 구현한 것이다. 즉, 과거에는 신호가 존재하지 않고, 어떤 시작 지점 이후에만 신호가 존재한다고 가정한 것이다.

2. printComplex 함수 정의

```
void printComplex(const complex<double>& z)
{
    double a = z.real();
    double b = z.imag();

    cout << a;

    if (b < 0) {
        cout << " - j" << -b;
    }
    else {
        cout << " + j" << b;
    }
}
```

3. finite length signal 계산

```
int main()
{
    // 1번 문제 신호:  $x[n] = a^n u[n]$ ,  $a=0.5$ ,  $n=0,1,2,\dots,50$ 
    //  $z = r * e^{jw}$ 
    double r = 0.9;
    double w = PI / 4.0; //  $\pi/4$ 

    complex<double> z = r * complex<double>(cos(w), sin(w));
    double a = 0.5;
    vector<double> x_exp(50);
    for (size_t n = 0; n < x_exp.size(); ++n) {
        x_exp[n] = pow(a, static_cast<int>(n)); //  $a^n$ 
    }

    complex<double> X_exp = zTransform1D(x_exp, z);

    cout << "finite length signal :  $X(z) =$  "; printComplex(X_exp);
    cout << endl << endl;
}
```

4. 이론식 계산

```
// 이론식 :  $X(z) = 1 / (1 - a*z^{-1})$ 
complex<double> z_inv = 1.0 / z;
complex<double> X_exp_theory = 1.0 / (1.0 - a * z_inv);

cout << "이론식 :  $X(z) =$  "; printComplex(X_exp_theory);
cout << endl << endl;

system("pause");
return 0;
}
```

결과

finite length signal : $X(z) = 1.161 - j0.751169$

이론식 : $X(z) = 1.161 - j0.751169$

계속하려면 아무 키나 누르십시오 . . . |

신호 $a^n u[n]$ 의 z-transform에서 ROC는 $|z| > |a|$ 이다. 이때 $a=0.5$, $z=0.9$ 로 설정했

	으므로 z가 ROC영역 안에 위치하게 된다. 따라서 z-transform의 값이 수렴하게 된다. 본 코드에서 finite length signal의 z-transform 항 n을 증가시킬수록 무한 등비급수의 이론 값에 수렴하게 되며, $n=0, 1, \dots, 50$으로 설정했을 때 이론 식 값과 동일한 결과가 나왔음을 확인할 수 있다.
--	--

Lab 2.

제출일	2025.12.17	학번	████████
전공	전자공학과	이름	차상훈

같이 한 사람: ██████████ ██████████

문 제	[2번 문제]: Apply an FIR Filter to a Grayscale Image(Lena_gray.bmp) and observe how the Image Changes
주 요 변 수	kernel, r_filtered, g_filtered, b_filtered
알 고 리 즘	1. firFilter2D 함수 정의 <pre>//FIR image filter 함수구현 vector<vector<int>> firFilter2D(const vector<vector<BYTE>>& input, const vector<vector<double>>& kernel) { int rows = input.size(); int cols = input[0].size(); int kH = kernel.size(); int kW = kernel[0].size(); int kCenterY = kH / 2; // 1 int kCenterX = kW / 2; // 1 vector<vector<int>> output(rows, vector<int>(cols, 0.0)); for (int i = 0; i < rows; i++) { for (int j = 0; j < cols; j++) { double sum = 0.0; //double 결과값 임시 버퍼 for (int m = 0; m < kW; m++) { for (int n = 0; n < kH; n++) { int yy = i + (m - kCenterY); int xx = j + (n - kCenterX); if (yy < 0 yy >= rows xx < 0 xx >= cols) { continue; } sum += kernel[m][n] * input[yy][xx]; } } output[i][j] = static_cast<int>(clamp(round(sum), 0.0, 255.0)); //반올림+정규화 } } return output; }</pre> 두 2차원 벡터를 받아 2차원 FIR filter 연산을 수행하는 함수를 구현한 것이다. 이 때 연산에 필요한 2차원의 입력 데이터들은 그냥 배열 타입이면 함수 작성 시에 입력의 열 길이를 고정해야 한다는 단점이 있다. 따라서 함수의 유연성이 떨어질 수 있다. 때문에 사진의 데이터를 담는 공간과 filter의 자료형을 벡터로 설정하여 함수에서 vector를 입력으로 받도록 구현했다. 또한 필터 연산을 거친 데이터를 rgb 표현 범위로 정규화 하는 작업은 cmath library의 반올림 함수 round()를 수행한 후 algorithm library의

clamp 함수로 숫자 범위를 제한했다. firFilter2D 함수의 반환 타입은 2차원 vector로, 위 코드에선 output이 반환된다.

2. 이미지 저장 코드

```
void main()
{
    // 이미지 저장 코드

    ifstream In_Image; // 파일 읽기
    In_Image.open(INPUT_FILE_NAME1, ios::in | ios::binary); // INPUT_FILE_NAME1 파일을 binary로 읽어옴

    int M = 64, N = 64; // 이미지의 크기 정의(단위:픽셀)
    BYTE* header = new BYTE[HEADERSIZE]; // 이미지의 헤더 정보를 담기 위한 공간 생성
    BYTE** image = new BYTE * [N]; // Lena_gray.bmp 파일의 이미지 데이터를 담기 위한 공간 생성
    vector<vector<BYTE>> r(N, vector<BYTE>(M, 0)); // 이미지 데이터 중 red에 대한 데이터를 담을 공간 생성
    vector<vector<BYTE>> g(N, vector<BYTE>(M, 0)); // 이미지 데이터 중 green에 대한 데이터를 담을 공간 생성
    vector<vector<BYTE>> b(N, vector<BYTE>(M, 0)); // 이미지 데이터 중 blue에 대한 데이터를 담을 공간 생성
    BYTE** result = new BYTE * [N]; // DFT와 IDFT를 통해 노이즈를 제거한 이미지 데이터를 담기 위한 공간 생성
    for (int i = 0; i < N; i++) { // 각각의 데이터를 담을 공간을 2차원의 형태로 만드는 과정(왜냐하면 이미지가 2
        image[i] = new BYTE[M * 3]; // 이미지 데이터를 담을 공간의 크기 : N * (M*3) (한 픽셀은 R,G,B 성분으로 0
        result[i] = new BYTE[M * 3]; // 노이즈를 제거한 이미지 데이터를 담을 공간의 크기 : N * (M*3) (출력 이미
    }

    In_Image.read((char*)header, HEADERSIZE); // 헤더 정보를 HEADERSIZE의 크기만큼 읽어서 header 변수에 저장
    for (int i = 0; i < N; i++) {
        In_Image.read((char*)image[i], 3 * M); // Lena_gray.bmp 파일의 이미지 데이터를 읽고 이를 image 변수에 저

    }

    int place;
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < M; j++) { // 이미지 데이터가 저장되어 있는 image 변수에서 R,G,B 값을 복사(실제 BMP
            place = 3 * j;
            b[i][j] = image[i][place]; // blue 데이터
            g[i][j] = image[i][place + 1]; // green 데이터
            r[i][j] = image[i][place + 2]; // red 데이터
        }
    }
}
```

기준에 제공되었던 코드에서 이미지의 데이터를 담을 r, g, b를 2차원 동적할당 배열에서 vector 자료형으로 변경해주었다. 이때 2차원 vector는 위 코드와 같이 작성하여 선언할 수 있다.

3. Vector형의 FIR filter

```
/* 작성해야 하는 코드 부분 ----- */

// 3x3 평균 필터 커널 (FIR)
vector<vector<double>> kernel1 =
{{1.0 / 9.0, 1.0 / 9.0, 1.0 / 9.0},
 {1.0 / 9.0, 1.0 / 9.0, 1.0 / 9.0},
 {1.0 / 9.0, 1.0 / 9.0, 1.0 / 9.0}};

vector<vector<double>> kernel2 =
{{ 0, -1.0, 0},
 {-1.0, 5.0, -1.0},
 { 0, -1.0, 0}};

vector<vector<double>> kernel3 =
{{ 0, -1.0, 0},
 {-1, 4.0, -1},
 { 0, -1.0, 0}};

// 필터 결과를 임시로 저장할 버퍼
vector<vector<int>> r_filtered(N, vector<int>(M, 0.0));
vector<vector<int>> g_filtered(N, vector<int>(M, 0.0));
vector<vector<int>> b_filtered(N, vector<int>(M, 0.0));
```

FIR filter를 2차원 vector로 설정하여 앞서 작성한 함수에 적용이 가능하도록 했다. 또한 필터 결과를 저장할 버퍼 또한 2차원 vector로 선언하여 앞서 작성한 함수의 결과 값을 저장하는 역할을 할 수 있도록 했다.

4. kernel 적용

```
// 2D FIR (컨볼루션) 수행: kernel1////////////////////////////////////
r_filtered = firFilter2D(r, kernel1);
g_filtered = firFilter2D(g, kernel1);
b_filtered = firFilter2D(b, kernel1);

// 이미지 생성 코드
for (int i = 0; i < N; i++) {
    for (int j = 0; j < M; j++) { // DFT와 IDFT를 통해 노이즈를 제거한 데이터를
        place = 3 * j;
        result[i][place] = b_filtered[i][j]; // gray 이미지는 R,G,B 값이 모두 같으
        result[i][place + 1] = g_filtered[i][j];
        result[i][place + 2] = r_filtered[i][j];
    }
}

ofstream Out1;
Out1.open("result1.bmp", ios::out | ios::binary); // 출력 파일 생성:kernel1
Out1.write((char*)header, HEADERSIZE); // 출력파일에 이미지의 헤더정보 작성
for (int i = 0; i < N; i++) {
    Out1.write((char*)result[i], 3.* M); // 노이즈를 제거하고 R,G,B이 하나의 픽셀이
}

////////////////////////////////////
```

kernel1

```
// 2D FIR (컨볼루션) 수행:kernel2////////////////////////////////////
r_filtered = firFilter2D(r, kernel2);
g_filtered = firFilter2D(g, kernel2);
b_filtered = firFilter2D(b, kernel2);

// 이미지 생성 코드
for (int i = 0; i < N; i++) {
    for (int j = 0; j < M; j++) { // DFT와 IDFT를 통해 노이즈를 제거한 데이터를 re
        place = 3 * j;
        result[i][place] = b_filtered[i][j]; // gray 이미지는 R,G,B 값이 모두 같으
        result[i][place + 1] = g_filtered[i][j];
        result[i][place + 2] = r_filtered[i][j];
    }
}

ofstream Out2;
Out2.open("result2.bmp", ios::out | ios::binary); // 출력 파일 생성:kernel2
Out2.write((char*)header, HEADERSIZE); // 출력파일에 이미지의 헤더정보 작성
for (int i = 0; i < N; i++) {
    Out2.write((char*)result[i], 3.* M); // 노이즈를 제거하고 R,G,B이 하나의 픽셀이
}

////////////////////////////////////
```

kernel2

```

// 2D FIR (컨볼루션) 수행:kernel3////////////////////////////////////
r_filtered = firFilter2D(r, kernel3);
g_filtered = firFilter2D(g, kernel3);
b_filtered = firFilter2D(b, kernel3);

// 이미지 생성 코드
for (int i = 0; i < N; i++) {
    for (int j = 0; j < M; j++) { // DFT와 IDFT를 통해 노이즈를 제거한 데이터를
        place = 3 * j;
        result[i][place] = b_filtered[i][j]; // gray 이미지는 R,G,B 값이 모두 같
        result[i][place + 1] = g_filtered[i][j];
        result[i][place + 2] = r_filtered[i][j];
    }
}

ofstream Out3;
Out3.open("result3.bmp", ios::out | ios::binary); // 출력 파일 생성:kernel3
Out3.write((char*)header, HEADERSIZE); // 출력파일에 이미지의 헤더정보 작성
for (int i = 0; i < N; i++) {
    Out3.write((char*)result[i], 3 * M); // 노이즈를 제거하고 R,G,B이 하나의 픽셀
}
////////////////////////////////////

```

kernel3

앞서 구현한 firFilter2D를 사용하여 이미지의 원본 데이터 값인 r, g, b와 kernel을 입력으로 넣은 결과를 rgb_filtered에 넣어주었다. 따라서 각 kernel들이 filter로서 원본 이미지 데이터에 적용된 결과 이미지가 각각 생성된다.

결과

1. kernel 1



원본



kernel1 적용 결과

kernel1의 성분은 전부 1/9이다. 이는 low-pass filter로서 동작하며 고주파 성분을 억제하는 효과가 있다. 영상/이미지 처리에선 일정 영역 데이터의 평균을 도출함으로써 smoothing효과를 내게 된다. 위 kernel1 적용 결과를 보면, 원본과 비교했을 때 이미지의 경계가 흐려진 것을 실제로 확인할 수 있다.

2. kernel2



원본



kernel2 적용 결과

```
vector<vector<double>> kernel2 =  
    {{ 0, -1.0, 0},  
     {-1.0, 5.0, -1.0},  
     { 0, -1.0, 0}};
```

kernel2

kernel2는 중심 5를 기준으로, 상하좌우가 -1, 대각 성분 0으로 이루어져 있다. 이는 중심 픽셀을 강조, 주변 픽셀과의 차이를 증폭한다. 따라서 원본 이미지에 보이는 경계들을 더 증폭하여 이미지 sharpening 역할을 한다. kernel2가 적용된 이미지에서, 원본 이미지와 비교했을 때 경계선이 더 뚜렷해진 것을 확인할 수 있다.

3. kernel3



원본



kernel3 적용 결과

```
vector<vector<double>> kernel3 =  
    {{ 0, -1.0, 0},  
     {-1, 4.0, -1},  
     { 0, -1.0, 0}};
```

kernel2

앞서 적용했던 filter들은 kernel의 전체 성분의 합이 1이었다. 전체 성분의 합은 DC 성분의 gain과 같은데, 따라서 이미지의 밝기를 filter 계수들의 합으로 조절할 수 있는 것이다. 전체 성분의 합이 1이면, 이미지의 전체적인 밝기를 유지함으로써 기존 이미지의 색깔 형태를 보존하게 된다.

반면, kernel3의 전체 성분 합은 0으로, DC 성분의 통과가 전혀 이루어지지 않는다. 원본 이미지에서 윤곽선이 뚜렷하지 않은 부분은 저주파 영역이므로 이러한 영역들이 filter를 거치면 픽셀의 데이터 값은 0이 나오게 된다. 위 필터 적용 결과 이미지를 보면, 윤곽선이 아닌 영역은 모두 검은색으로 나타나며, 원본 사진에 있던 물체의 경계 선만 표현된 것을 알 수 있다.

느
낀
점

영상이나 이미지가 디지털 세상에서 어떻게 가공되는지 이해할 수 있었다. 특히 이러한 2D filter의 원리나 적용 방법들은 디스플레이라는 매체가 처음 개발되었을 때 이미지 가공의 필요성에 의해 개발이 되었을 것인데, 기존에 통신 신호처리에서 noise 감소, 신호 증폭 등의 방법론들이 이미지의 가공에서 또한 적용될 수 있다는 사실이 매우 흥미

	롭 다.
--	------